LAB 03 Widget tree, layouts and adaptive UI

Unit 2 • 3-hour guided laboratory • CLO-5

Learning goal. Compose readable screens using Row, Column, Expanded, Stack and LayoutBuilder without common overflow errors.

Core concepts. StatelessWidget, widget composition, constraints, responsive breakpoints, Card, SafeArea, MediaQuery, Flutter image/icon resources.

Before you start. Create two new copies of a Lab 03 project so each code example can replace lib/main.dart.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab03
cd mad_lab03
```

Solved task 1 | Responsive faculty profile page

Problem and objective. Build a reusable profile card that places portrait and detail side by side on wide screens but stacks content on narrow phones.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: ProfilePage()));

class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Faculty profile')),
      body: SafeArea(child: LayoutBuilder(builder: (context, size) {
        final wide = size.maxWidth >= 520;
        final portrait = Container(
          width: wide ? 140 : 100, height: 120,
          decoration: BoxDecoration(
            color: Colors.teal.shade100,
            borderRadius: BorderRadius.circular(16),
          ),
          child: const Icon(Icons.person, size: 75),
        );
        final details = const Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          mainAxisSize: MainAxisSize.min,
          children: [
            Text('Muhammad Jamshaid Iqbal',
                style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
            SizedBox(height: 8),
            Text('Department of Computer Science'),
            Text('COMSATS University Islamabad – Sahiwal'),
            SizedBox(height: 8),
            Chip(label: Text('Mobile Application Development')),
          ],
        );
        return SingleChildScrollView(
          padding: const EdgeInsets.all(20),
          child: Card(child: Padding(
            padding: const EdgeInsets.all(16),
            child: wide
                ? Row(children: [portrait, const SizedBox(width: 18),
                    Expanded(child: details)])
                : Column(children: [portrait, const SizedBox(height: 16),
                    details]),
          )),
        );
      })),
```

```
    );
  }
}
```

How the solution works. LayoutBuilder exposes available width from the parent. The breakpoint changes the child arrangement, Expanded prevents the details from exceeding the horizontal Row, and SingleChildScrollView protects against small screens and large accessibility text.

Expected output. A narrow Android phone shows the icon above the profile; a tablet or landscape width may show a side-by-side card.

Check your work. Rotate the device; test with larger system font size and verify that no black/yellow overflow stripes appear.

Solved task 2 | Responsive product dashboard with adaptive grid

Problem and objective. Show mock inventory cards with different grid density depending on available screen width.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: InventoryPage()));

class InventoryPage extends StatelessWidget {
  const InventoryPage({super.key});
  static const items = [
    ('Laptop', Icons.laptop, 129000),
    ('Mouse', Icons.mouse, 2200),
    ('Keyboard', Icons.keyboard, 4900),
    ('Phone', Icons.smartphone, 68000),
    ('Headphones', Icons.headphones, 5500),
    ('Tablet', Icons.tablet, 87000),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Responsive inventory')),
      body: LayoutBuilder(builder: (context, size) {
        final columns = size.maxWidth >= 700 ? 4 :
            size.maxWidth >= 420 ? 3 : 2;
        return GridView.builder(
          padding: const EdgeInsets.all(12),
          gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
            crossAxisCount: columns, crossAxisSpacing: 12,
            mainAxisSpacing: 12, childAspectRatio: .90,
          ),
          itemCount: items.length,
          itemBuilder: (context, i) {
            final item = items[i];
            return Card(child: Padding(
              padding: const EdgeInsets.all(8),
              child: Column(mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  Icon(item.$2, size: 38, color: Colors.teal),
                  const SizedBox(height: 8),
                  Text(item.$1, textAlign: TextAlign.center),
                  Text('Rs. ${item.$3}', textAlign: TextAlign.center),
                ],
              ),
            ));
          },
        );
      }),
    );
  }
}
```

How the solution works. The grid delegate computes cell sizes from count and aspect ratio. Two, three or four columns are chosen by container width rather than a fixed screen assumption; builder avoids constructing off-screen cells unnecessarily.

Expected output. Six cards render in an adaptive two-to-four-column grid without requiring image downloads.

Check your work. Check portrait and landscape orientation and explain which threshold gets selected at 420 and 700 pixels.

Scenario-based practice — unsolved

Scenario 1. Accessible noticeboard. Create a noticeboard that remains usable with 200% font scaling and narrow screens. Specify scroll strategy, minimum touch target and semantic labels.

Scenario 2. Cafeteria product cards. Design a price card that switches layouts between portrait and landscape while preserving content order and accessible labels.

Technical viva — justify your reasoning

1. What exactly are box constraints and why can Row overflow?
2. When should Expanded be used instead of Flexible?
3. Why is LayoutBuilder often more precise than reading global screen width?
4. How does ListView.builder avoid eager work?
5. Does a Stack automatically become scrollable? Explain.
6. What testing proves a layout is accessible for text scaling?

Submission and self-check

Capture phone portrait and landscape, record chosen grid counts and show flutter analyze without errors.

CSC303 / NOVEMBER 2026 REVISION

LAB 03 EXTENSION | Repeatable accessibility checks

Responsive layout is not only avoiding a RenderFlex overflow at the default font size. Students should verify that the interface remains usable when text is enlarged, when screen readers are enabled and when orientation changes.

Mandatory manual checks

Test	How to perform it	Pass condition
Large text	Raise device font/display size; repeat at an enlarged scale	Text can be read; actions remain available; no clipping
Portrait and landscape	Rotate phone; test the narrowest supported viewport	No overflow; navigation and buttons remain reachable
Screen-reader labels	Enable TalkBack; navigate without visual targeting	Actionable controls have meaningful spoken labels
Touch interaction	Try controls with one hand and on a small phone	Interactive targets are easily distinguishable and operable
Theme and contrast	Inspect light/dark themes and error states	Meaning conveyed by more than color alone

Example Semantics for an icon-only action

```
IconButton(
  tooltip: 'Delete saved note',
  icon: const Icon(Icons.delete_outline),
  onPressed: deleteNote,
)
```

Prefer meaningful built-in labels and tooltips; do not wrap every widget in duplicate Semantics nodes. Use the Accessibility Inspector or device screen reader for verification. For animations, check reduced-motion behavior in Lab 14.

Student exercise

- In Lab 03 Task 1, demonstrate narrow and wide layouts and capture screen images at normal and enlarged text.
- Record one actual accessibility failure and the source-code change that resolves it.
- Attach the relevant before/after evidence to the lab repository; avoid including any private device data.